

Distributed Key-Value Database Architecture

A Non-Technical Guide to How It Works

1. Executive Summary

This document describes the design and operation of our Distributed Key-Value Database in simple terms. Imagine a digital storage system designed like a set of lockers where each box has a unique label (a 'Key') and contains an item (a 'Value'). To make it fast, secure, and reliable, we run multiple database servers together, replicate changes in real-time, write diaries of every action so we never lose data, and use standard languages to talk to clients. Below is a breakdown of these mechanisms using everyday analogies.

2. The Core Concept: The Locker Room (Key-Value Store)

In traditional databases, data is stored in complex, massive spreadsheets linked together. In a Key-Value database, we simplify this: it acts like a locker room. Each piece of information has a specific label (the Key, like 'user_id_101') that opens a locker containing the item (the Value, like 'Alice Smith').

Operations are simple:

- GET: Look up a label and retrieve the item inside (takes $O(1)$ average time, meaning it's almost instantaneous).
- PUT: Put a new item in a locker under a specific label (or replace what is already there).
- DELETE: Empty a locker and remove its label.

3. Distributed Replication (The Boss & The Assistants)

Running a database on just one computer is risky. If that computer crashes, the business stops. To prevent this, our database is 'Distributed' -- meaning it runs on multiple computers coordinating together. We use a 'Leader-Follower' (Primary-Secondary) model:

- The Leader (The Boss): This is the main server that accepts instructions to add, update, or delete data. When the Leader changes any information, it immediately broadcasts the instructions to all connected Followers.
- The Followers (The Assistants): These servers mirror the Leader. They only allow reading of data (looking inside lockers) and block any direct writes from the outside. If the Leader goes offline, a Follower holds a perfect duplicate of the data, so the system stays online.

4. Fault Tolerance: Write-Ahead Logging & Snapshots

Computers can lose power suddenly. If that happens, any information kept only in memory disappears. We solve this using two main tools:

- Write-Ahead Logging (The Diary): Before we put an item in a locker, we write down our action in an append-only diary on the hard drive (e.g., 'Put Alice in locker 101'). Because writing a quick sentence to a diary is extremely fast, we don't slow down the database. If power cuts out, the server reboots, reads the diary from page one, and reconstructs the locker room exactly as it was.
- Snapshot Compaction (Clean Inventories): Over time, the diary gets too long and takes too much space. To clean it, we periodically write down a single summary sheet of all active lockers (a 'Snapshot') and throw away the old diary. This keeps the database fast and clean.

5. High Performance: Multi-threading & SQL Queries

- Multi-threading (Multiple Receptionists): If thousands of users connect at once, a single receptionist at the front desk would cause a massive line. Our database spawns multiple worker threads (virtual receptionists). Each incoming customer is handed to a dedicated receptionist so they are served concurrently with zero delay.
- SQL (Speaking the Standard Language): Databases use a language called SQL (Structured Query Language). Even though our database is a key-value store, we built a translator (SQL Parser) under the hood. Non-technical business

Distributed Key-Value Database Architecture

A Non-Technical Guide to How It Works

reporting tools can send standard queries like 'SELECT value FROM kv WHERE key = ...' and our database translates it on the fly to look inside our lockers.

Quick Features Comparison

Feature Name	Business Purpose	Analogy Used
Key-Value Indexing	Low-Latency Lookups	A Locker Room with numbered boxes
Primary-Secondary Replication	Fault Tolerance / Backups	The Boss (writes) and Assistants (reads)
Write-Ahead Log (WAL)	Crash Durability	Logging operations in a diary book
Snapshot Compaction	Storage Optimization	Replacing diaries with a clean inventory list
SQL Query Parser	Tool Compatibility	Standard language translator

Designed and built in C++ with native Winsock2 TCP layers.